Deploying a Serverless Web Application Using AWS App Runner

Scenario Description:

A startup is exploring modern cloud-native options for deploying containerized applications with minimal infrastructure management. The team chooses AWS App Runner to deploy a web application directly from source code or a container image with built-in support for scalability, load balancing, and HTTPS out of the box.

Scenario Steps:
Setting Up the App Runner Environment:
Prepare the Application

Develop a simple web app using Node.js, Flask, or any framework of your choice.

Option 1: Push the source code to a GitHub repo.
Option 2: Build a container image and push it to Amazon ECR Public or Private (ECR Guide).

1. Deploy Using AWS App Runner Console
2. Go to App Runner in the AWS Management Console.
3. Choose the deployment source:
4. Source Code Repository (e.g., GitHub)
5. Container Registry (Amazon ECR)
6. Configure runtime settings: port, environment variables, and CPU/memory.
7. Enable automatic deployments if using GitHub.
8. Configure Auto Scaling and Load Balancing
9. App Runner automatically provisions a load balancer and scales containers based on request volume.
10. You can customize concurrency and auto scaling policies.
11. Set up Custom Domain and HTTPS (Optional)
12. Add a custom domain.

13. App Runner provides free HTTPS certificates via AWS Certificate Manager.

Note
- You can choose either Option 1 or Option 2 based on your preference—both are valid for deploying your application with AWS App Runner.

AWS APP-Runner

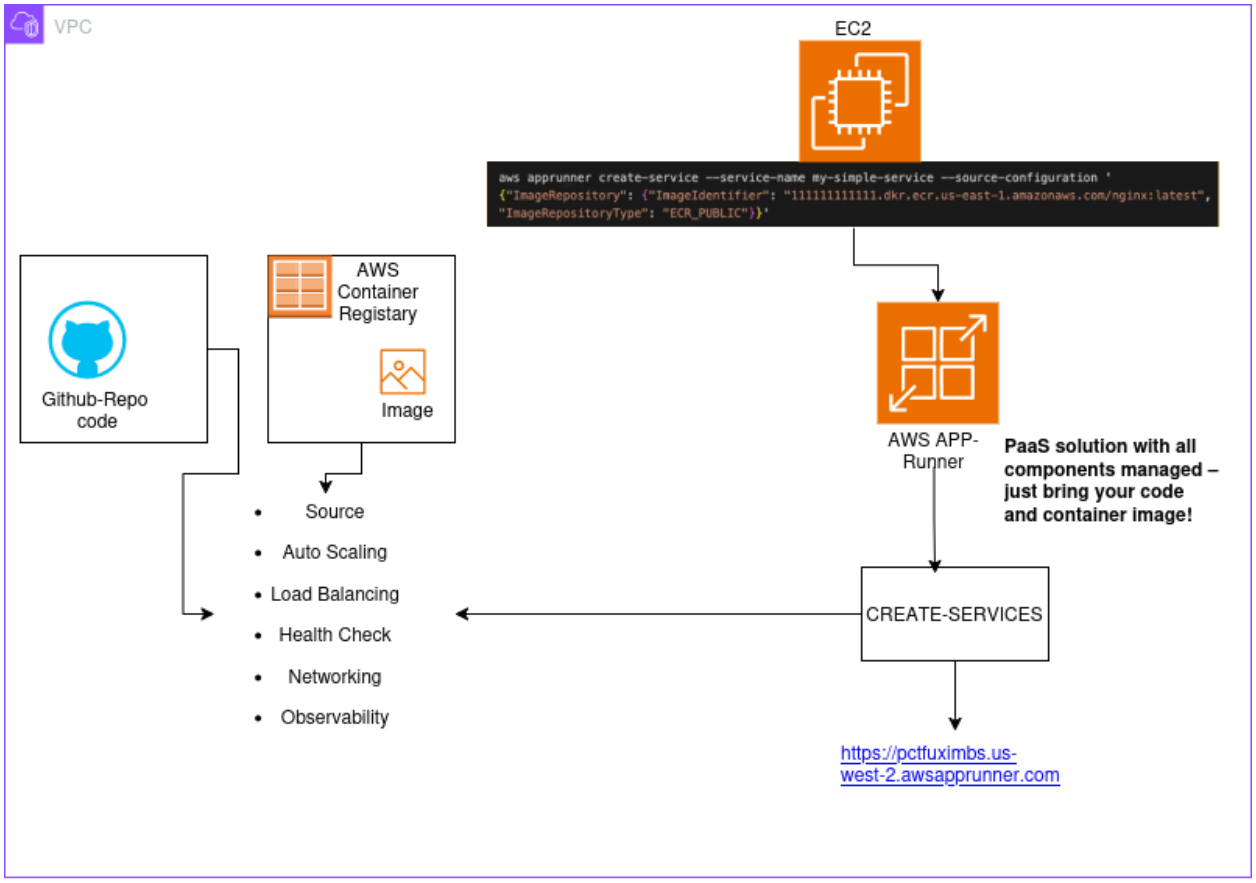
Fully managed service for deploying containerized web apps and APIs

In this project,

we learned that AWS App Runner can deploy web applications in two different ways:

Option 1: App Runner can deploy from GitHub → automatic.
Option 2: App Runner can deploy from ECR (container image) → More control.

We imagine Architecture like that:



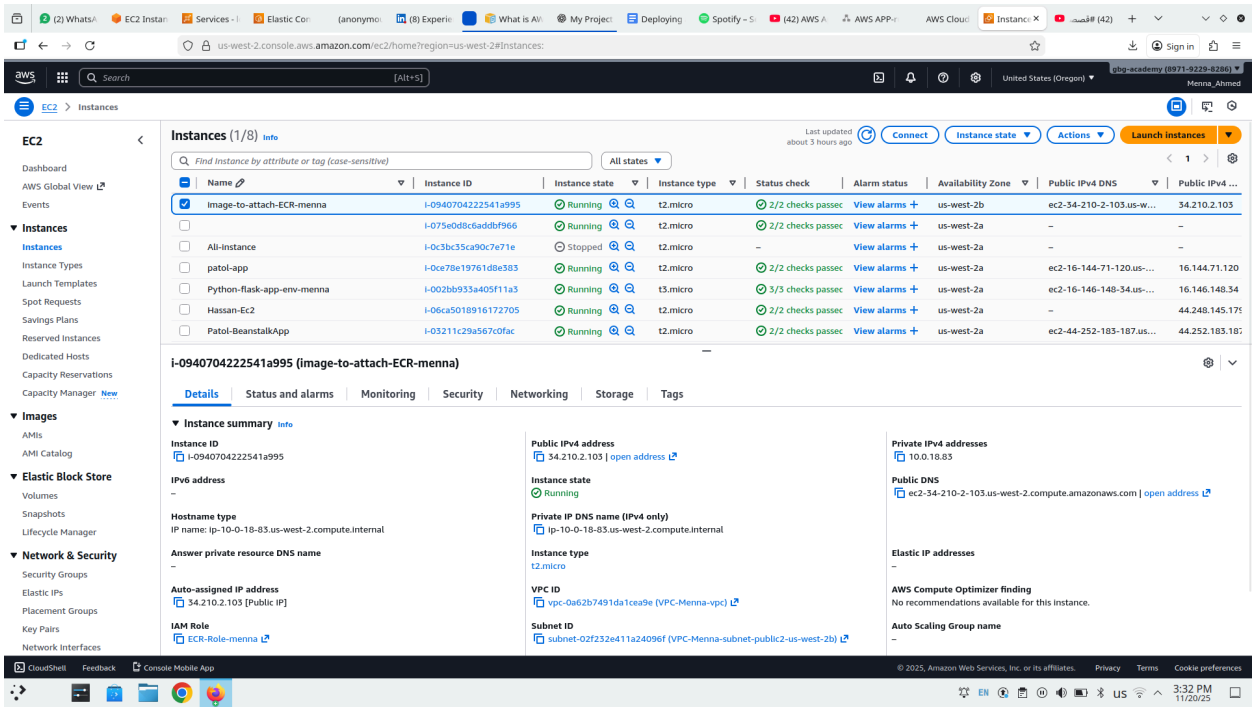
Technical part :

Option 2: App Runner can deploy from ECR (container image) → More control.

1- Create and prepare an EC2 instance:

- Install Docker

- Pull or build an application image (e.g., nginx)



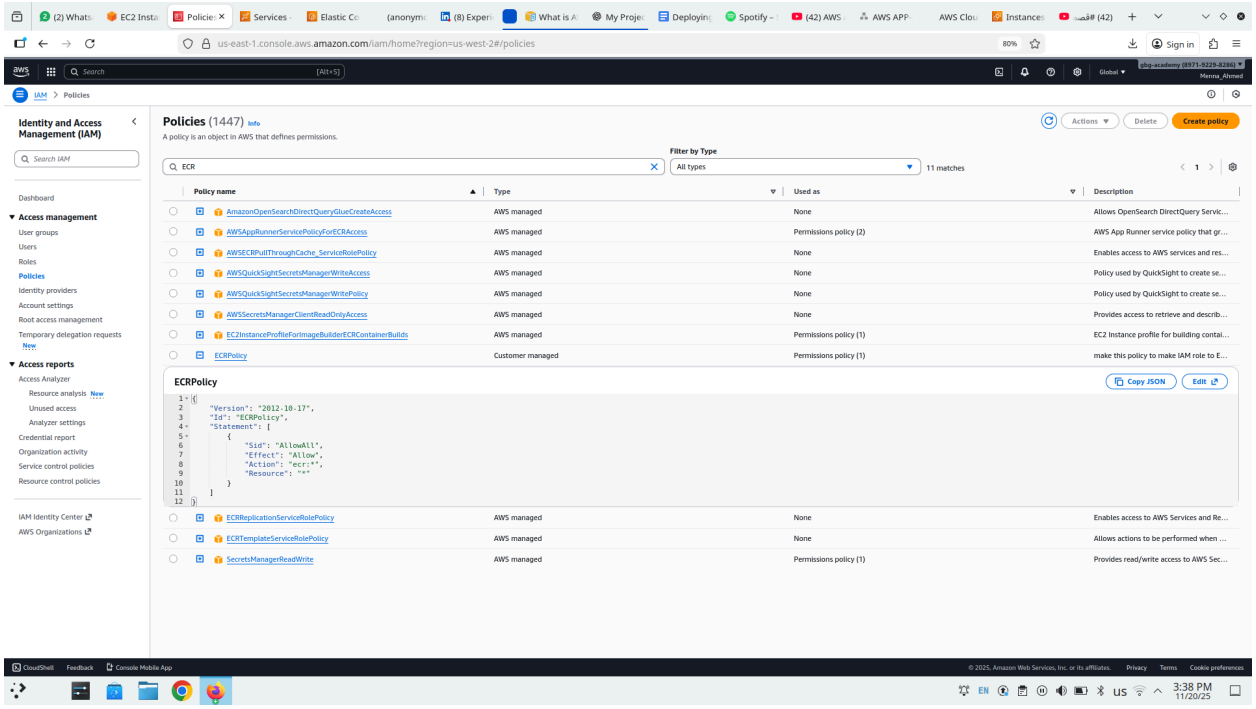
2- Connect To EC2 to run Command

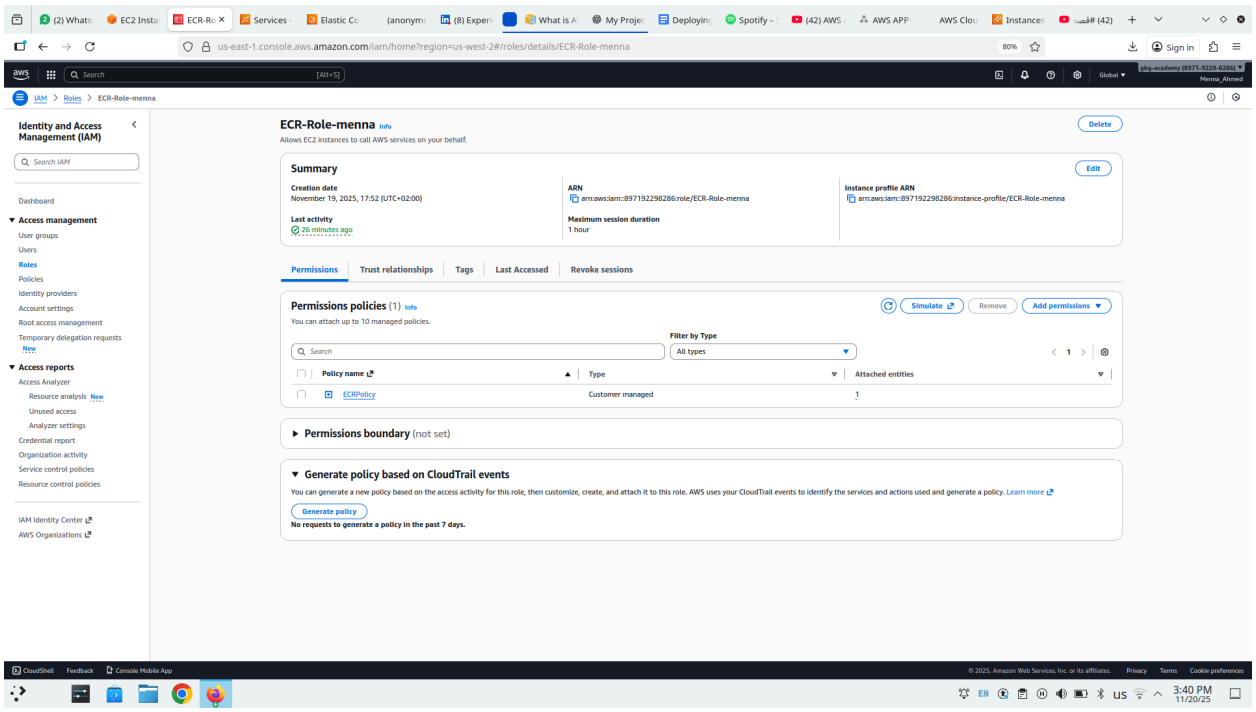
Run the following commands on EC2:

- sudo su
- yum update -y
- yum install docker -y
- systemctl enable docker.service
- systemctl start docker.service
- docker images
- docker pull nginx
- docker images

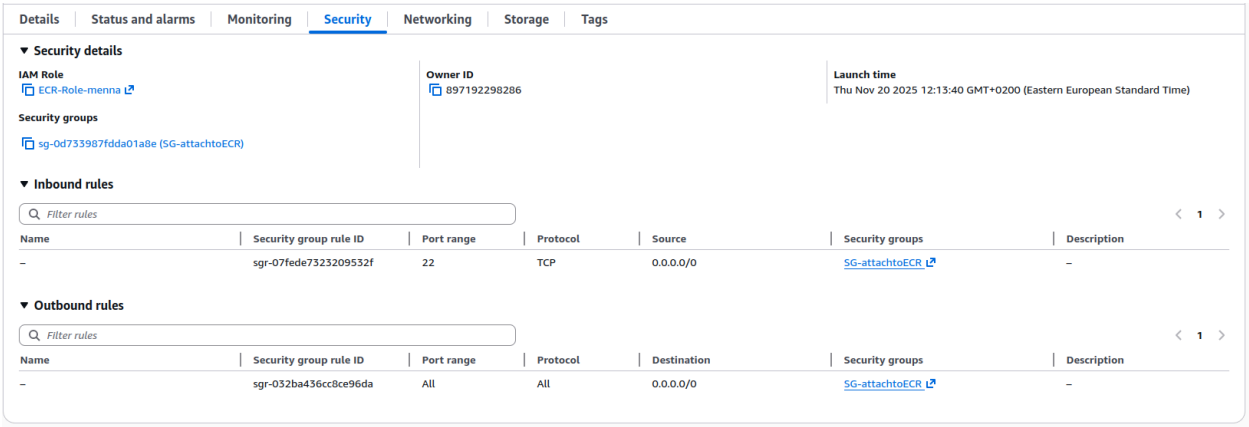
3- Create Policy and IAM Role

To attach this Policy to IAM Role I will Creating


us-east-1.console.aws.amazon.com/iam/home?region=us-west-2#/policies
IAM > Policies
Identity and Access Management (IAM)
Policies (1447) Info
A policy is an object in AWS that defines permissions.
Filter by Type
ECR
All types
11 matches
Policy name
Type
Used as
Description
AmazonOpenSearchDirectQueryGlueCreateAccess
AWS managed
None
Allows OpenSearch DirectQuery Servic...
AWSAppRunnerServicePolicyForECRAccess
AWS managed
Permissions policy (2)
AWS App Runner service policy that gr...
AWSECRPullThroughCache_ServiceRolePolicy
AWS managed
None
Enables access to AWS services and res...
AWSQuickSightSecretsManagerWriteAccess
AWS managed
None
Policy used by QuickSight to create se...
AWSQuickSightSecretsManagerWritePolicy
AWS managed
None
Policy used by QuickSight to create se...
AWSSecretsManagerClientReadOnlyAccess
AWS managed
None
Provides access to retrieve and describ...
EC2InstanceProfileForImageBuilderECRContainerBuilds
AWS managed
Permissions policy (1)
EC2 Instance profile for building contai...
ECRPolicy
Customer managed
Permissions policy (1)
make this policy to make IAM role to E...
ECRPolicy
Copy JSON
Edit
"Version": "2012-10-17",
"Id": "ECRPolicy",
"Statement": [
"Sid": "AllowAll",
"Effect": "Allow",
"Action": "ecr:*",
"Resource": "*"
ECRReplicationServiceRolePolicy
AWS managed
None
Enables access to AWS Services and Re...
ECRTemplateServiceRolePolicy
AWS managed
None
Allows actions to be performed when ...
SecretsManagerReadWrite
AWS managed
Permissions policy (1)
Provides read/write access to AWS Sec...


us-east-1.console.aws.amazon.com/iam/home?region=us-west-2#/roles/details/ECR-Role-menna
IAM > Roles > ECR-Role-menna
Identity and Access Management (IAM)
ECR-Role-menna Info
Allows EC2 instances to call AWS services on your behalf.
Delete
Summary
Edit
Creation date
November 19, 2025, 17:52 (UTC+02:00)
Last activity
26 minutes ago
ARN
arn:aws:iam::897192298286:role/ECR-Role-menna
Maximum session duration
1 hour
Instance profile ARN
arn:aws:iam::897192298286:instance-profile/ECR-Role-menna
Permissions
Trust relationships
Tags
Last Accessed
Revoke sessions
Permissions policies (1) Info
You can attach up to 10 managed policies.
Simulate
Remove
Add permissions
Filter by Type
All types
Policy name
Type
Attached entities
ECRPolicy
Customer managed
1
Permissions boundary (not set)
Generate policy based on CloudTrail events
You can generate a new policy based on the access activity for this role, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. Learn more
Generate policy
No requests to generate a policy in the past 7 days.

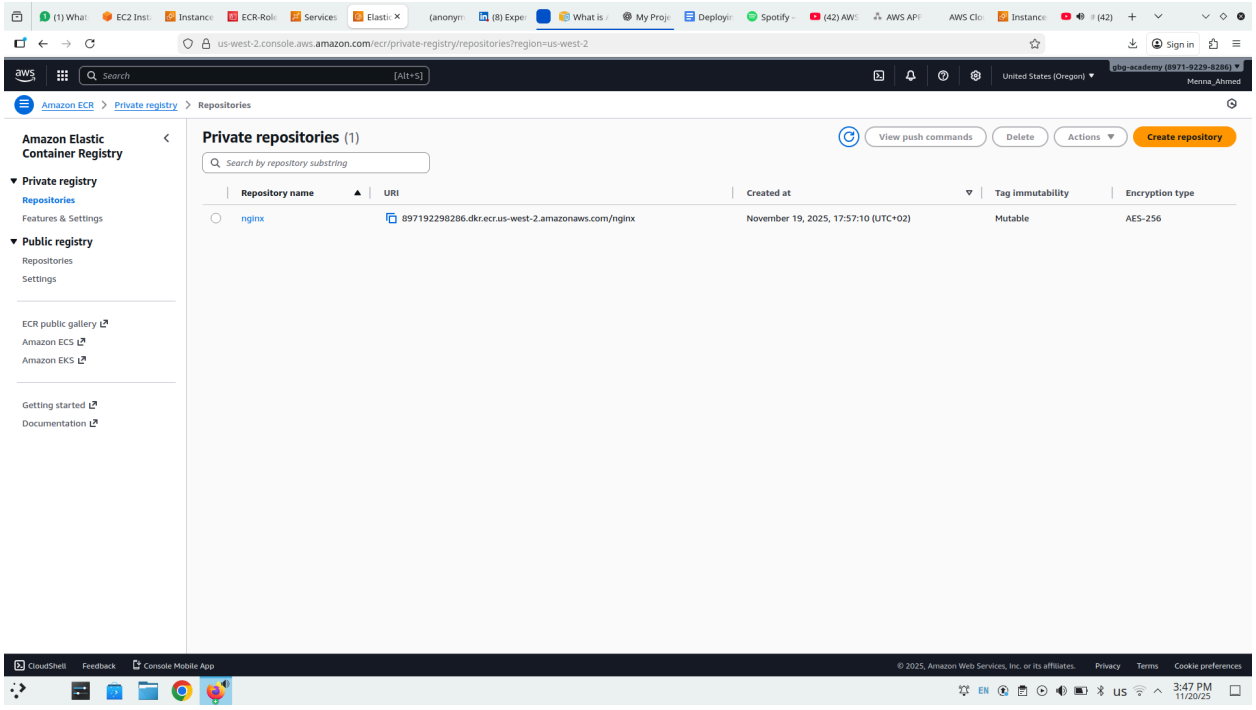
4- Make Modify to Add IAM Role to EC2 Instance



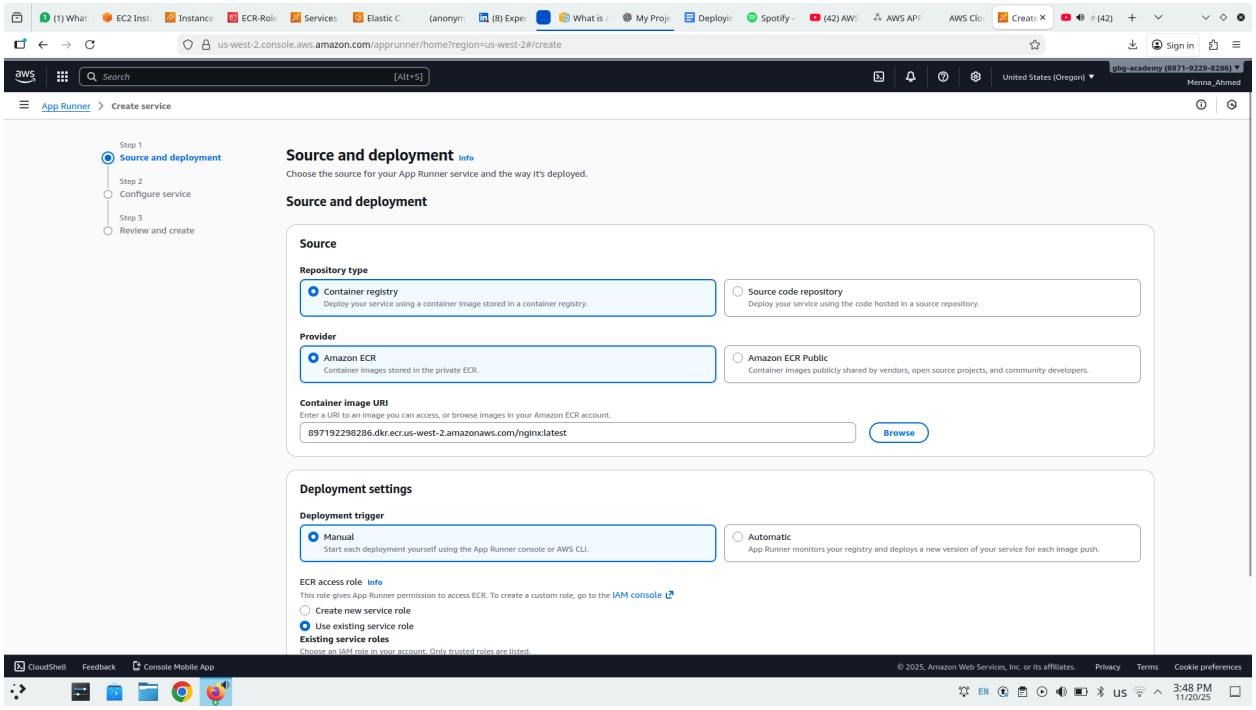
5- Go back again to Connect EC2 and write these command :

- aws ecr create-repository --repository-name nginx --region us-west-2
- docker tag nginx:latest 8971922982861.dkr.ecr.us-west-2.amazonaws.com/nginx:latest
- aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin8971922982861.dkr.ecr.us-west-2.amazonaws.com
- docker push 8971922982861.dkr.ecr.us-west-2.amazonaws.com/nginx:latest

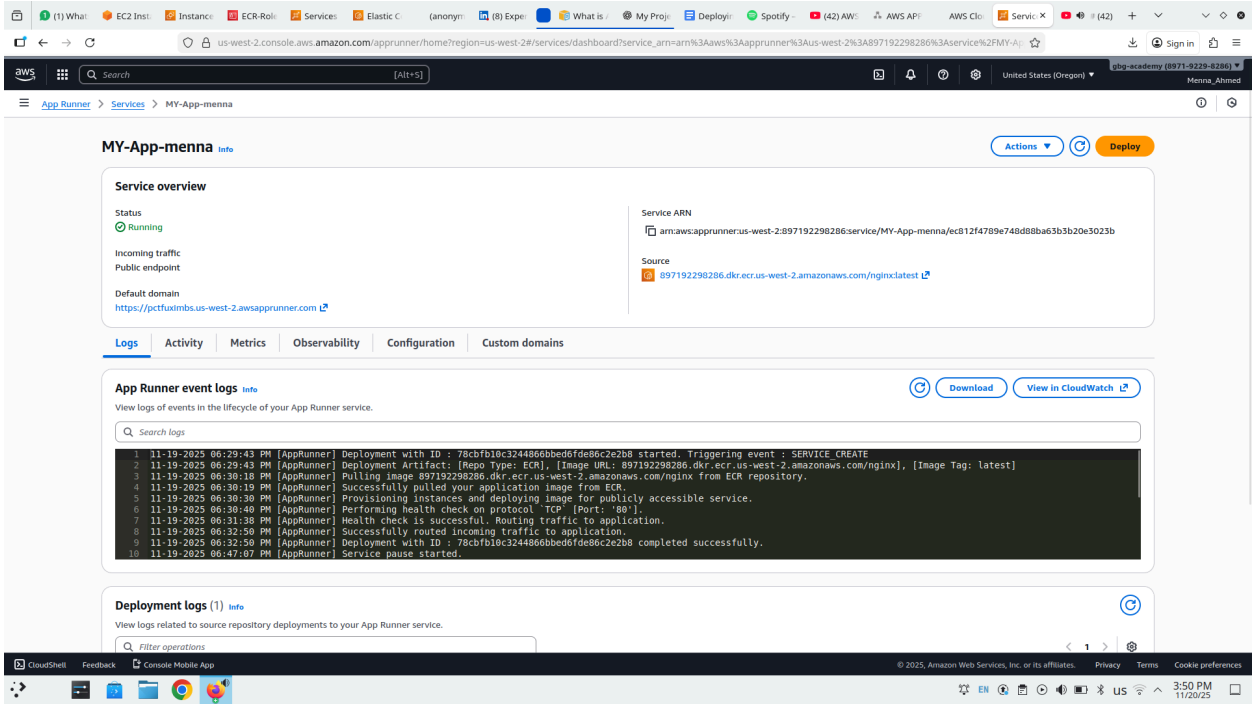
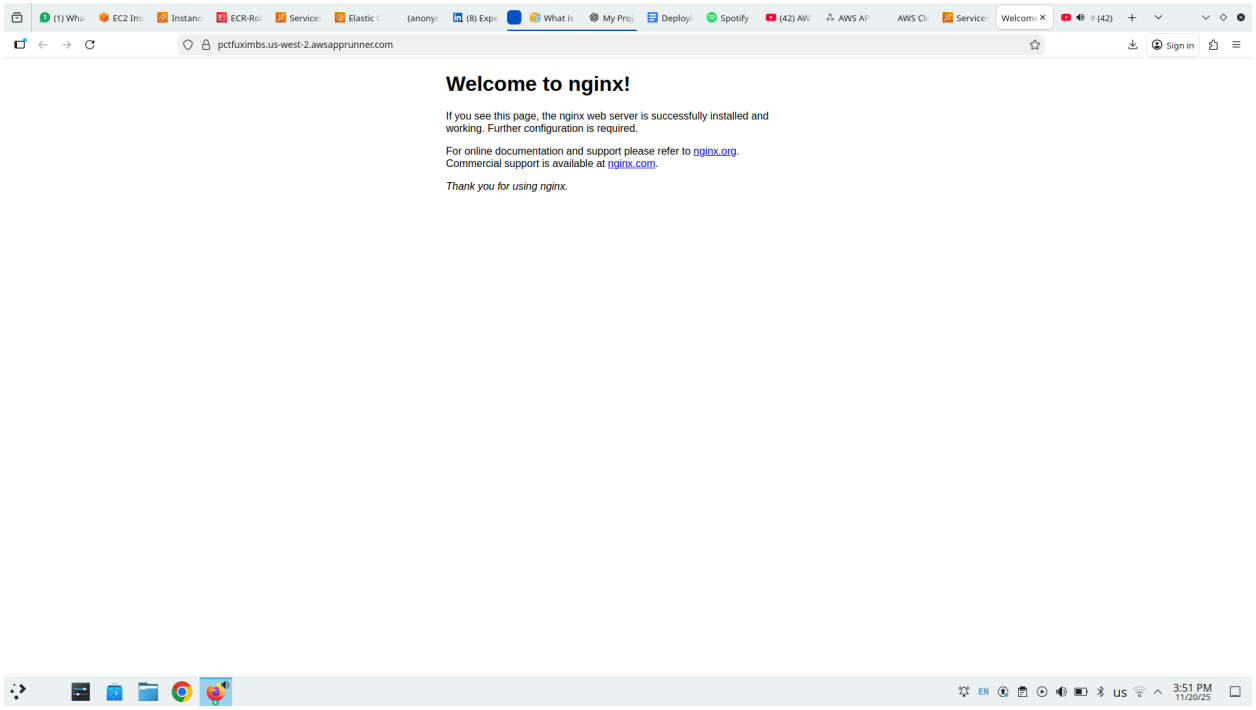
6- You will see nginx is downloading on ECR



7- Create App-runner services and put nginx



8- create and open the domain is created and you will see nginx



Open domain https://pctfuximbs.us-west-2.awsapprunner.com